

POWER BI

Interview Guide & Professional Handbook

[DAX Functions](#) | [Architecture](#) | [Data Modeling](#) | [Interview Q&A](#)
[Professional Workflow](#) | [Installation](#) | [Data Sources](#) | [Best Practices](#)

DTank54 Group | A1 English Level
Complete Reference for Power BI Learners

18 SECTIONS

TABLE OF CONTENTS

Section 1: Essential Power BI Terminology

All key terms explained in simple English for beginners

Section 2: What is Power BI?

Understanding the platform and its ecosystem

Section 3: Power BI Architecture

How Power BI works behind the scenes

Section 4: DAX Basics

What is DAX and why it matters

Section 5: DAX Functions - 6 Categories

All important DAX functions grouped

Section 6: CALCULATE Deep Dive

The most powerful DAX function explained

Section 7: Filter Context vs Row Context

The two most important concepts

Section 8: Iterator Functions (X-Functions)

Functions that loop through rows

Section 9: Data Modeling (Star Schema)

How to organize your data model

Section 10: Date Table

Why every report needs a date table

Section 11: 15 Interview Q&A

Most asked Power BI interview questions

Section 12: Quick Tips & Shortcuts

Speed up your daily work

Section 13: DAX Patterns

Common formulas you will use in every project

Section 14: Quick Reference Card

All key facts on one page

Section 15: Professional Workflow Sequence

Step-by-step: how experts do projects

Section 16: Free Power BI Desktop Installation Tutorial

Install and start learning today

Section 17: Realistic Data Sources for Practice

Where to find free real data

Section 18: Best Practices & Advanced Approach

Rules professionals follow

SECTION 1 | Essential Power BI Terminology

Before you start learning Power BI, you need to understand the key words and terms. Every subject has its own vocabulary. Medicine has medical terms. Law has legal terms. Power BI also has many special terms. If you do not know what these words mean, you will feel lost even if you understand the concepts. This section explains every important Power BI term in simple, clear English. Read this section first. Then the rest of this guide will be much easier to understand.

Think of this section as a dictionary for Power BI. You can come back to this section anytime you see a word you do not understand. All terms are listed in alphabetical order with examples.

Core Concepts

Dashboard

A dashboard is a single page that shows the most important numbers and charts from your data. It is like the main screen of a car: it shows your speed, fuel level, and warnings at a glance. In Power BI, a dashboard usually has 4-8 key visuals: KPI cards (big numbers), a bar chart, a line chart, and maybe a map. The goal is to give managers a quick overview without making them click through many pages.

Example: A Sales Dashboard shows Total Revenue, Total Profit, Profit Margin %, and a chart of Revenue by Country. The CEO opens this dashboard every morning to check the health of the business.

Report

A report is a collection of one or more pages with visuals (charts, tables, maps, cards). A report is more detailed than a dashboard. A dashboard is one page with summary numbers. A report can have many pages, each showing a different aspect of the data. For example, Page 1 shows the summary, Page 2 shows product details, Page 3 shows geographic analysis. You build reports in Power BI Desktop.

Example: A Sales Report has 4 pages: Executive Summary, Product Analysis, Geographic Analysis, and Time Analysis. Each page has multiple charts and slicers.

Visual (Visualization)

A visual is any chart, graph, table, map, or card that shows your data visually. When you create a bar chart, a line chart, or a map in Power BI, each one is called a "visual." Power BI has many types of visuals: bar charts, line charts, pie charts, maps, tables, cards, gauges, KPI indicators, scatter plots, treemaps, and more. You create visuals by dragging fields from the Fields pane onto the canvas.

Example: A bar chart showing Revenue by Country is one visual. A map showing Sales by Country is another visual. A card showing Total Revenue is also a visual.

Slicer

A slicer is a special type of visual that acts as a filter. It is like a filter button that the user can click. When the user clicks a value in the slicer, all other visuals on the page change to show only data for that selected value. Slicers make reports interactive. Without slicers, the report shows all data all the time. With slicers, users can explore the data by clicking different options.

Example: You add a Country slicer with values: USA, Canada, France, Germany, Mexico. When the user clicks "France," all charts on the page show only data for France. When they click "USA," everything changes to show USA data.

Data and Model Terms

Table

A table is a collection of data organized in rows and columns, just like an Excel spreadsheet. In Power BI, each table has a name and contains columns. For example, a "Sales" table might have columns like Date, Product, Revenue, and Profit. You can have multiple tables in one Power BI file. Each table represents one type of data: one table for products, one table for customers, one table for sales transactions.

Example: A "Products" table has 7 rows (7 products) and columns: ProductID, ProductName, Category, Color, Price.

Column

A column is a vertical list of values in a table. Each column has a name and a data type (like text, number, date, or currency). A table can have many columns. For example, in a "Sales" table, you might have columns like: OrderDate, ProductName, Quantity, UnitPrice, TotalAmount. Each column stores one type of information. The "Quantity" column stores only numbers. The "ProductName" column stores only text.

Example: In a Sales table, "Revenue" is a column that contains numbers: \$100, \$250, \$500, etc. "Country" is a column that contains text: USA, France, Germany, etc.

Row

A row is one horizontal line of data in a table. Each row represents one record or one transaction. For example, in a Sales table, one row might be: Date=01/15/2024, Product=Montana, Country=USA, Revenue=\$2500, Profit=\$800. That is one complete record. If you have 700 sales transactions, your table has 700 rows.

Example: Row 1 = {Date: 2014-01-01, Product: Montana, Segment: Government, Country: France, Sales: \$1000, Profit: \$400}. This is ONE sale transaction.

Field

A field is another word for a column. When you see "Fields pane" in Power BI, it shows all the columns from all your tables. Power BI uses the word "field" to mean either a column from a table or a measure that you created. When you drag a field onto a chart, you are telling Power BI to use that column or measure in the visual.

Example: In the Fields pane, you will see your table name (like "Sales") and under it, all the fields (columns): Date, Product, Country, Revenue, Profit, and any measures you created.

Fact Table

A fact table is the main table in your data model that contains the numbers and transactions. It stores the "facts" or events of your business: sales amounts, quantities, costs, and dates. A fact table usually has many rows (thousands or millions) because it records every transaction. It also contains foreign keys (ID columns) that connect it to dimension tables. The name "fact" means "things that actually happened."

Example: A "FactSales" table has 700 rows. Each row is one sale: SaleID, DateID, ProductID, CustomerID, UnitsSold, Revenue, Cost, Profit.

Dimension Table (Dim Table)

A dimension table contains descriptive information about the entities in your business. While a fact table stores numbers, a dimension table stores descriptions. For example, a Product dimension table stores product names, categories, colors, and brands. A Customer dimension table stores customer names, cities, and segments. A Date dimension table stores year, month, quarter, and day names. Dimension tables usually have fewer rows than fact tables.

Example: "DimProduct" table has 7 rows (one per product): ProductID, ProductName, Category, Subcategory, Color, Brand, ListPrice.

Star Schema

A star schema is a way of organizing your tables that looks like a star when you draw it. One large fact table sits in the center. Several smaller dimension tables surround it, connected by lines (relationships). The name comes from the star shape. This is the best-practice way to organize data in Power BI because it makes DAX simple, reports fast, and the model easy to understand.

Example: FactSales (center) connected to DimProduct, DimCustomer, DimDate, DimCountry (around it). It looks like a star with 4 points.

Relationship

A relationship is a connection between two tables based on a shared column. For example, the Sales table has a ProductID column, and the Products table also has a ProductID column. You create a relationship between them so Power BI knows that ProductID 1 in Sales means "Montana" in the Products table. Relationships allow you to use fields from one table in visuals that use data from another table.

Example: Sales[ProductID] is connected to DimProduct[ProductID]. When you drag DimProduct[ProductName] into a chart that shows SUM(Sales[Revenue]), Power BI uses the relationship to know which product name belongs to each sale.

Cardinality

Cardinality describes the type of relationship between two tables. There are three main types in Power BI. "Many-to-One" (most common): many sales rows can belong to one product (one product has many sales). "One-to-One": each row in one table matches exactly one row in the other table (like an employee and their security badge). "Many-to-Many": many rows in one table match many rows in the other (special cases, avoid if possible).

Example: The relationship between Sales and Products is Many-to-One because one product can have many sales, but each sale belongs to only one product.

DAX Terms

Measure

A measure is a dynamic calculation that you create using DAX. Unlike a regular column that has a fixed value for each row, a measure does not store any data. Instead, it calculates a result on demand every time a user interacts with the report. When you change a slicer or click on a chart, the measure recalculates automatically. Measures are the most powerful feature of Power BI. They answer business questions like "What is total revenue?" or "What is the profit margin?" Measures appear with a calculator icon in the Fields pane.

Measure Examples: Total Revenue = SUM(Sales[Amount]) Profit Margin % = DIVIDE([Total Profit], [Total Revenue], 0) YTD Sales = TOTALYTD(SUM(Sales[Amount]), Date[Date])

Calculated Column

A calculated column is a new column that you add to an existing table using a DAX formula. Unlike a measure, a calculated column stores a value for every row in the table. It is computed once when the data loads and then saved in memory. Use calculated columns when you need a value that belongs to a specific row (like adding a "Full Name" column that combines First Name and Last Name). Avoid calculated columns when a measure can do the same job, because calculated columns use more memory.

Calculated Column Example: Full Name = Customer[FirstName] & " " & Customer[LastName]
(This adds a new column where each row has the full name)

Measure vs Calculated Column (Key Difference)

This is the most confusing topic for beginners. Here is the simple rule: A Calculated Column gives a value for EACH ROW and stores it in memory. A Measure gives ONE result for the whole table (or a filtered portion) and recalculates every time. Think of it like this: a calculated column is like writing an answer in every cell of an Excel spreadsheet. A measure is like writing a SUM formula that changes when you filter the data. Always try to use Measures first. They are more flexible and use less memory.

Calculated Column: Stores one value per row. Uses more memory. Does NOT change with filters. Measure: Calculates one result for the whole context. Uses less memory. CHANGES with filters and slicers.

DAX (Data Analysis Expressions)

DAX is the formula language of Power BI. It stands for Data Analysis Expressions. You use DAX to write formulas that calculate business metrics. DAX looks similar to Excel formulas but is much more powerful. In Excel, formulas work cell by cell. In DAX, formulas work with entire tables and columns. DAX also has advanced concepts like filter context and row context that do not exist in Excel. Learning DAX is the most important skill for Power BI developers.

Simple DAX: Total Sales = SUM(Sales[Amount]) Advanced DAX: YTD = CALCULATE(SUM(Sales[Amount]), DATESYTD(Date[Date]))

Filter Context

Filter context is the set of filters that decide WHICH rows of data are included in a calculation. Every time you add a slicer, click on a chart, or use CALCULATE, you are changing the filter context. Think of filter context as a "net" that catches only certain rows of data. If you have a Country slicer set to "France," the filter context catches only rows where Country = France. All measures on that page will only calculate using those French rows.

Example: If you have a bar chart showing Revenue by Product, and the Country slicer is set to "USA," the filter context is Country = USA. The chart shows revenue for each product, but only for USA sales.

Row Context

Row context means DAX is looking at one specific row at a time, not the whole table. This happens in two situations: (1) When you create a Calculated Column, DAX goes through each row one by one and calculates a value for that row. (2) When you use an X-function like SUMX, FILTER, or RANKX, DAX loops through rows one by one and evaluates an expression for each row. Row context does NOT automatically filter other tables. That is a common mistake beginners make.

Example: In a Calculated Column "Tax" = Sales[Amount] * 0.20, the row context means: for THIS specific row, take the Amount and multiply by 0.20. DAX does this for every row, one at a time.

Power Query Terms

Power Query

Power Query is the data cleaning and transformation tool inside Power BI. It is where you prepare your data before it enters the Power BI data model. You use Power Query to connect to data sources, remove unwanted columns, filter rows, change data types, merge tables, and fix data quality issues. Power Query uses a language called "M" behind the scenes, but you usually do not need to write M code manually because there are buttons and menus for every operation. The key rule: Power Query runs ONCE during data load. DAX runs every time the user interacts with the report.

Example: You load an Excel file that has 20 columns. You only need 5 of them. In Power Query, you delete the 15 unnecessary columns, fix the date format, remove rows with errors, and then load the clean data into Power BI.

M Language

M is the programming language behind Power Query. When you perform operations in the Power Query Editor (like filtering, removing columns, merging tables), Power Query automatically generates M code in the background. You can see this code by clicking "Advanced Editor" in Power Query. Most of the time, you do not need to write M code manually because the graphical interface does everything. But knowing M is useful for advanced transformations that the menus cannot do.

Example of M code: `let Source = Excel.Workbook(File.Contents("C:\data\sales.xlsx")),
FilteredRows = Table.SelectRows(Source, each [Sales] > 1000) in FilteredRows`

Data and Connection Terms

Data Source

A data source is the place where your data lives before you bring it into Power BI. Data sources can be many things: Excel files, CSV files, SQL Server databases, web pages, SharePoint lists, Azure databases, Salesforce, Google Analytics, APIs, and more than 100 other options. Power BI can connect to all of these. The data source is the "home" of your data. Power BI copies the data from the source (Import mode) or reads it directly (DirectQuery mode).

Examples of Data Sources: An Excel file on your computer, a SQL Server database in your company, a web API that provides stock prices, a SharePoint folder with daily reports.

Import Mode

Import mode means Power BI copies ALL the data from your data source into its own memory. After the import, the data lives inside the .pbix file. This mode is fast for report viewing because the data is already in memory. But you need to "refresh" the data periodically to get updates from the source. Import mode is the default and works best for datasets up to 1-2 GB.

Example: You import an Excel file with 100,000 rows. Power BI copies all 100,000 rows into its memory. When the Excel file changes, you need to click "Refresh" to get the new data.

DirectQuery Mode

DirectQuery mode means Power BI does NOT copy the data. Instead, it sends queries directly to the source database every time a user interacts with the report. This means the data is always live and up-to-date. However, it can be slower than Import mode because every click sends a query to the database. Use DirectQuery when your dataset is too large to fit in memory or when you need real-time data.

Example: Your company has a SQL Server database with 500 million rows of sales data. This is too big to import into Power BI. You use DirectQuery mode so Power BI queries the SQL Server directly.

Refresh

Refresh means updating the data in your Power BI report to match the latest data in the source. There are two types of refresh. "Manual Refresh" means you click the Refresh button yourself. "Scheduled Refresh" means Power BI Service automatically refreshes the data at times you choose (for example, every day at 8 AM). Refresh is only needed in Import mode. In DirectQuery mode, data is always live so refresh is not needed.

Example: You have a Power BI report connected to an Excel file that the finance team updates every Friday. You set Scheduled Refresh to run every Saturday at 6 AM so the report shows the latest numbers on Monday morning.

Visual and Report Terms

KPI (Key Performance Indicator)

A KPI is a number that shows how well the business is performing against a goal. In Power BI, KPIs are usually shown as large card visuals on the top of a dashboard. Common KPIs are: Total Revenue, Profit Margin %, Year-over-Year Growth %, Customer Count, and Order Count. A KPI is more than just a number. It is a number with context: it tells you whether you are winning or losing.

Example: Revenue = \$5.2M (good if target was \$5M, bad if target was \$8M). YoY Growth = 15% (good if positive, bad if negative). These are KPIs.

Axis

An axis is the horizontal or vertical line of a chart that shows categories or values. The X-axis (horizontal) usually shows categories like products, months, or countries. The Y-axis (vertical) usually shows values like revenue amounts or profit percentages. When you build a chart, you drag fields onto the Axis and Values areas to tell Power BI what to show on each axis.

Example: In a bar chart of Revenue by Country: X-axis (Axis) = Country names (USA, France, Germany...). Y-axis (Values) = Revenue amounts (\$100K, \$250K, \$300K...).

Legend

A legend is a small box on a chart that explains what each color or symbol means. When you drag a field into the Legend area of a chart visual, Power BI uses different colors to represent different values of that field. For example, if you put "Product Category" in the Legend of a bar chart, each category gets a different color, and the legend shows which color means which category.

Example: A stacked bar chart shows Revenue by Country, and the Legend shows "Bicycles" in blue and "Accessories" in orange. You can see how much each category contributes to each country total.

Tooltip

A tooltip is a small popup box that appears when you hover your mouse over a chart element (like a bar or a dot). It shows detailed information about that specific element. For example, when you hover over a bar in a Revenue by Country chart, the tooltip might show: Country = France, Revenue = \$250,000, Profit = \$80,000, Units = 1,200. You can customize tooltips in Power BI to show the most useful information.

Example: Hover over a bar in the chart and see: "Montana, Q1 2014, Revenue: \$15,000, Profit: \$5,200, Units Sold: 300."

Bookmark

A bookmark saves the current state of a report page: which slicers are selected, which visuals are visible, and how they are filtered. You can create multiple bookmarks for the same page and switch between them using buttons. Bookmarks are useful for creating presentations within Power BI. For example, you can create a bookmark called "Revenue View" and another called "Profit View" that show different charts and filters.

Example: You create a bookmark that shows only 2024 data with specific product filters. Another bookmark shows all data with no filters. Users click buttons to switch between these views.

Drill-Down

Drill-down means going from a summary level to a detail level. For example, you start with a chart showing Revenue by Year. When you double-click on the "2024" bar, the chart "drills down" to show Revenue by Quarter for 2024. Double-click on Q1, and it drills down to Revenue by Month for Q1 2024. This hierarchy navigation (Year > Quarter > Month) is called drill-down. It allows users to explore data at different levels of detail without needing separate charts.

Example: Start at Year level (2024 = \$10M). Click to drill down to Quarter level (Q1=\$2M, Q2=\$3M, Q3=\$2.5M, Q4=\$2.5M). Click Q2 to drill down to Month level (Apr=\$1M, May=\$1M, Jun=\$1M).

Cross-Filtering

Cross-filtering happens when you click on one visual and it automatically filters all other visuals on the same page. For example, if you click on the "France" bar in a Revenue by Country chart, all other charts on the page (like Revenue by Product, Revenue Trend) will also filter to show only France data. This is one of the most powerful features of Power BI. It allows users to explore data naturally by clicking around the report.

Example: You have 3 charts on one page: Revenue by Country, Revenue by Product, and Revenue Trend. Click "Germany" in the first chart. The second chart shows only German product revenue. The third chart shows only the German revenue trend.

File and Sharing Terms

.pbix File

A .pbix file is the file format of Power BI Desktop reports. When you create a report in Power BI Desktop and save it, the file has a .pbix extension. This file contains everything: your data model, all DAX measures, all report pages and visuals, and all formatting. It is like a complete package of your work. You can open .pbix files only in Power BI Desktop (not in Excel or other programs). The file can be large if your dataset is big.

Example: My_First_Report.pbix is a file that contains the Financial Sample data model, 13 DAX measures, 4 report pages, and 20 visuals. File size might be 5 MB.

Power BI Service (Cloud)

Power BI Service is the online version of Power BI. It is also called "Power BI in the cloud." After you build a report in Power BI Desktop, you publish it to Power BI Service so other people can view it through a web browser. Power BI Service also handles scheduled data refresh, user permissions, and workspace management. The free version has limitations. The Pro version (\$10/month per user) is needed for most business features.

Example: You build a report in Power BI Desktop on your computer. Then you click "Publish" to upload it to Power BI Service. Now your manager can open their web browser, go to app.powerbi.com, and view the report.

Workspace

A workspace is a shared area in Power BI Service where you and your team can collaborate on reports. Think of it like a folder in the cloud. You can create a workspace called "Sales Team," upload your sales reports, and give access to your team members. Everyone in the workspace can see and interact with the reports. Workspaces help organize reports by team or project.

Example: Your company has three workspaces: "Sales Reports," "Finance Reports," and "HR Reports." Each team publishes their reports to their own workspace. Team members only see their own workspace.

Row-Level Security (RLS)

Row-Level Security is a feature that controls which data each user can see in a report. For example, a sales manager for France should only see French sales data, not data from other countries. RLS automatically filters the data based on who is logged in. This is very important for enterprise reports where different users should see different data. RLS is set up using roles and DAX filters.

Example: You create a role called "France Manager" with the filter: `DimCountry[Country] = "France"`. When the France manager opens the report, they only see data for France, even if the report has data for 20 countries.

Read This First!

If you are new to Power BI, read this entire terminology section before moving to the other sections. Knowing these terms will make everything else in this guide much easier to understand. Come back to this section anytime you see a word you do not recognize.

SECTION 2 | What is Power BI?

Power BI is a business intelligence tool made by Microsoft. It helps people see and understand their data. With Power BI, you can connect to many data sources, make beautiful reports, and share them with your team. Companies all over the world use Power BI every day to make better decisions based on data instead of guesses.

Think of Power BI as a bridge between raw data and clear answers. You have data in Excel, SQL, websites, or cloud systems. Power BI takes all this data, cleans it, connects it, and shows it in visual charts and dashboards. This means you do not need to be a programmer to use it. The goal of Power BI is simple: help anyone understand data quickly and easily.

The Three Parts of Power BI

Part	What It Does	Cost
Power BI Desktop	The main tool on your computer. You build reports here. This is where you write DAX, make charts, and connect data.	FREE
Power BI Service (Cloud)	The online version. You upload your reports here and share them with other people through a web browser.	Free / Pro (\$10/month)
Power BI Mobile	The phone app. You can view your dashboards on your phone or tablet anytime.	Free

What Can You Do With Power BI?

- Connect to Excel files, SQL databases, web pages, APIs, and 100+ other data sources
- Clean and transform data with Power Query (no coding needed)
- Create interactive charts: bar charts, line charts, maps, tables, cards, and many more
- Write DAX formulas to calculate custom business metrics (measures)
- Build dashboards that update automatically when data changes
- Share reports with your team through Power BI Service or Teams
- Ask questions in natural language with AI-powered Q&A

Power BI vs Other Tools

Feature	Power BI	Tableau	Excel
Price	Free Desktop + Paid Cloud	Expensive	Part of Office
DAX / Calculations	Very strong (DAX)	Good (LOD)	Limited
Data Connections	100+ sources	80+ sources	Limited
Learning Curve	Easy to start	Medium	Easy to start
Best For	Microsoft ecosystem	Visual analytics	Quick analysis
Sharing	Easy (Cloud)	Server needed	Email / OneDrive

SECTION 3 | Power BI Architecture

Understanding the architecture of Power BI is important for interviews and for real work. Power BI has many parts that work together like a system. Each part has a clear job. When you understand the architecture, you can explain to managers and clients how Power BI works from start to finish.

The Big Picture

Layer	What Happens Here	Tools Used
1. Data Layer	Data lives here. It can be in databases, files, cloud, or APIs. This is where your raw data is stored.	SQL Server, Azure, Excel, SharePoint, Web API
2. Data Integration	Data is collected, cleaned, and transformed. Power Query connects and shapes the data.	Power Query (M language), Dataflows
3. Data Modeling	Create relationships between tables. Build the star schema. This is the engine of your report.	Power Pivot, DAX, Model view
4. Visualization	Build charts, tables, maps, and dashboards. This is what users see and interact with.	Report view, Visualizations pane
5. Sharing Layer	Publish reports to the cloud. Set permissions. Users view reports in browser or mobile.	Power BI Service, Workspaces, Apps

Power Query vs DAX vs Power Pivot

Tool	Language	Job	When to Use
Power Query	M language	Clean and transform data before it loads into the model	When you need to filter rows, change column types, merge tables, or add new columns from existing data
DAX	DAX language	Create calculations on the loaded data model	When you need new measures, calculated columns, or dynamic aggregations like year-to-date totals
Power Pivot	DAX + Model	Build the data model and relationships	When you need to connect tables, define relationships, and organize the star schema

Data Refresh Types

- Manual Refresh: You click the Refresh button to update data right now
- Scheduled Refresh: Power BI Service automatically refreshes data at times you choose (every day at 8 AM)
- DirectQuery: No refresh needed. Power BI talks to the database directly. Always live data.
- Incremental Refresh: Only refresh new or changed data (not all data). Faster for large datasets.

Interview Tip

"What is the difference between Power Query and DAX?" Answer: "Power Query cleans the data BEFORE it loads into the model. DAX creates calculations AFTER the data is already in the model." This answer shows deep understanding.

SECTION 4 | DAX Basics

DAX stands for Data Analysis Expressions. It is the formula language of Power BI. You use DAX to create calculations that go beyond simple counting and summing. DAX is very powerful and learning it well will make you stand out in interviews and at work.

DAX looks similar to Excel formulas, but it is very different behind the scenes. In Excel, each cell calculates independently. In DAX, calculations work with entire tables and columns. DAX also has concepts like filter context and row context, which do not exist in Excel. This makes DAX more powerful but also more complex.

Two Types of DAX Calculations

Type	What It Is	Where It Lives	Memory
Calculated Column	A new column you add to a table. Each row gets a value calculated from other columns in the same row.	In the table (like any other column)	Uses more memory (stored in the table)
Measure	A dynamic calculation. It changes based on filters, slicers, and what the user selects in the report.	In the model (not in any table)	Uses less memory (calculated on demand)

CALCULATED COLUMN example:

Profit = Sales[Amount] - Sales[Cost]

-- This creates a new column. Each row gets its own profit value.

MEASURE example:

Total Sales = SUM(Sales[Amount])

Average Sales = AVERAGE(Sales[Amount])

-- Measures recalculate every time the user changes a filter or slicer.

Golden Rule

Always try to use Measures instead of Calculated Columns. Measures use less memory and give more flexible results. Only use Calculated Columns when you need the value in a row filter or as a relationship key.

Basic DAX Syntax

Every DAX formula follows a simple pattern. Understanding the syntax is the first step to writing good DAX:

Measure Name = FUNCTION(Table[Column])

Examples:

Total Revenue = SUM(Sales[Revenue])

Product Count = DISTINCTCOUNT(Product[Name])

Max Price = MAX(Product[Price])

Orders Over 100 = COUNTROWS(FILTER(Sales, Sales[Amount] > 100))

SECTION 5 | DAX Functions - 6 Categories

DAX has hundreds of functions. But you do not need to memorize all of them. In real work and interviews, you will use the same 30-40 functions again and again. This section groups the most important functions into 6 categories so you can learn them in a structured way.

Category 1: Aggregation Functions

Function	What It Does	Example
SUM()	Adds all values in a column	SUM(Sales[Amount])
AVERAGE()	Calculates the mean (average)	AVERAGE(Sales[Amount])
MIN()	Finds the smallest value	MIN(Sales[Amount])
MAX()	Finds the largest value	MAX(Sales[Amount])
COUNT()	Counts numeric values	COUNT(Sales[ID])
COUNTA()	Counts non-blank values	COUNTA(Customer[Name])
DISTINCTCOUNT()	Counts unique values	DISTINCTCOUNT(Sales[Product])
COUNTROWS()	Counts rows in a table	COUNTROWS(Sales)

Category 2: Filter Functions

Function	What It Does	Example
FILTER()	Returns a table with filtered rows	FILTER(Sales, Sales[Amount] > 100)
ALL()	Removes all filters from a table or column	ALL(Sales[Region])
ALLEXCEPT()	Removes all filters except specified ones	ALLEXCEPT(Sales, Sales[Year])
CALCULATE()	Changes the filter context for a measure	CALCULATE(SUM(Sales), Sales[Year]=2024)
HASONEVALUE()	Checks if a column has exactly one value	HASONEVALUE(Product[Category])
SELECTEDVALUE()	Returns the single value if only one exists	SELECTEDVALUE(Product[Category])
VALUES()	Returns unique values of a column	VALUES(Customer[City])

Category 3: Time Intelligence Functions

Function	What It Does	Example
TOTALYTD()	Year-to-date total	TOTALYTD(SUM(Sales[Amount]), Date[Date])
TOTALQTD()	Quarter-to-date total	TOTALQTD(SUM(Sales[Amount]), Date[Date])

Function	What It Does	Example
TOTALMTD()	Month-to-date total	TOTALMTD(SUM(Sales[Amount]), Date[Date])
SAMEPERIODLASTYEAR())	Same period last year	CALCULATE(SUM(Sales), SAMEPERIODLASTYEAR(Date[Date]))
DATEADD()	Shift dates by interval	DATEADD(Date[Date], -1, MONTH)
STARTOFMONTH()	First day of month	STARTOFMONTH(Date[Date])
ENDOFMONTH()	Last day of month	ENDOFMONTH(Date[Date])
DATESYTD()	Year to date dates	DATESYTD(Date[Date])

Category 4: Relationship Functions

Function	What It Does	Example
RELATED()	Gets a value from a related table (many-to-one)	RELATED(Product[Category])
RELATEDTABLE()	Gets rows from a related table (one-to-many)	RELATEDTABLE(Sales)
USERELATIONSHIP()	Activates an inactive relationship	CALCULATE(MEASURE, USERELATIONSHIP(T1[C], T2[C]))
CROSSFILTER()	Changes filter direction of a relationship	CROSSFILTER(Sales[ProductID], Product[ID], BOTH)

Category 5: Logical Functions

Function	What It Does	Example
IF()	Returns one value if true, another if false	IF(Sales[Amount] > 100, "High", "Low")
SWITCH()	Checks many conditions (like multiple IF)	SWITCH(Product[Cat], "A", 1, "B", 2, 3)
AND()	Returns TRUE if all conditions are TRUE	IF(AND(A > 10, B > 10), "Both High", "No")
OR()	Returns TRUE if any condition is TRUE	IF(OR(A > 100, B > 100), "One High", "No")
COALESCE()	Returns first non-blank value	COALESCE(Table1[Col], Table2[Col], 0)
ISBLANK()	Checks if a value is blank	IF(ISBLANK(Sales[Discount]), 0, Sales[Discount])
ISNUMBER()	Checks if a value is a number	IF(ISNUMBER(Value), Value, 0)

Category 6: Text and Information Functions

Function	What It Does	Example
CONCATENATE()	Joins text strings together	CONCATENATE(FirstName, " ", LastName)
LEFT() / RIGHT()	Gets characters from left or right	LEFT(ProductCode, 3)
SEARCH()	Finds text within text	SEARCH("phone", Description)
FORMAT()	Converts value to text with format	FORMAT(Date, "MMM-YYYY")
LEN()	Returns the number of characters	LEN(Product[Name])
UPPER() / LOWER()	Converts to uppercase or lowercase	UPPER(Customer[Name])
TRIM()	Removes extra spaces from text	TRIM(Customer[Name])
REPLACE()	Replaces part of a text string	REPLACE(Phone, 1, 3, "+994")

SECTION 6 | CALCULATE Deep Dive

CALCULATE is the most important function in DAX. It is the answer to many interview questions. If you can master CALCULATE, you can solve 80% of all DAX problems. CALCULATE changes the filter context of a measure. This means it can modify what data is included in a calculation, even after the user has applied filters.

Basic Syntax

```
CALCULATE(Expression, Filter1, Filter2, ...)
```

The first argument is always the expression (what you want to calculate). After that, you can add one or more filter arguments. Each filter changes which rows are included in the calculation.

Simple Examples

```
-- Total sales for year 2024 only:
Sales 2024 = CALCULATE(SUM(Sales[Amount]), Sales[Year] = 2024)

-- Total sales for a specific product:
Phone Sales = CALCULATE(SUM(Sales[Amount]), Product[Name] = "Phone")

-- Combine multiple filters:
Sales 2024 Phones = CALCULATE(
    SUM(Sales[Amount]),
    Sales[Year] = 2024,
    Product[Category] = "Electronics"
)
```

CALCULATE with ALL()

One of the most powerful combinations is CALCULATE with ALL(). The ALL() function removes existing filters, and then CALCULATE applies new ones. This is how you create "ignore all filters and show me this specific thing" calculations:

```
-- Total of ALL sales (ignore all filters from slicers):
Total All Sales = CALCULATE(SUM(Sales[Amount]), ALL(Sales))

-- Percentage of total:
% of Total = DIVIDE(
    SUM(Sales[Amount]),
    CALCULATE(SUM(Sales[Amount]), ALL(Sales))
)
```

Interview Must-Know

"What does CALCULATE do?" Answer: "CALCULATE is the only function in DAX that can modify the filter context. It evaluates its expression in a modified filter context created by its filter arguments. This makes it the most versatile and important function in DAX."

SECTION 7 | Filter Context vs Row Context

Filter Context and Row Context are the two most important concepts in DAX. Understanding them is the difference between a beginner and an expert. Many students find this confusing at first, but with clear examples it becomes simple.

What is Filter Context?

Filter Context is the set of filters that are applied to a calculation at any given moment. These filters come from many sources: slicers, visual-level filters, page-level filters, report-level filters, and the CALCULATE function. Filter Context determines WHICH rows are included in a calculation. It comes from the OUTSIDE (from the report design and user selections).

Example:

-- If you have a slicer on Year = 2024:

Total Sales = SUM(Sales[Amount])

-- The filter context here is: Year = 2024 (from slicer)

-- Only sales from 2024 are included in the sum

What is Row Context?

Row Context exists when DAX is looking at one row at a time. This happens in two situations: in a Calculated Column (where DAX calculates a value for each row) and in Iterator functions like SUMX, AVERAGEX, FILTER (where DAX loops through rows one by one). Row Context answers the question: "For this specific row, what is the value?"

Example:

-- In a Calculated Column, DAX looks at ONE row at a time:

Tax = Sales[Amount] * 0.20

-- For THIS specific row: take the Amount and multiply by 0.20

Key Differences

Aspect	Filter Context	Row Context
What it does	Determines WHICH rows are visible	Looks at ONE row at a time
Where it comes from	Slicers, filters, CALCULATE	Calculated columns, X-functions
How it works	Filters the data first, then calculates	Goes row by row and calculates
Example function	CALCULATE changes filter context	SUMX, FILTER create row context

Common Mistake

Many students try to use SUM(Sales[Amount]) * 0.20 inside a measure and expect it to work like a column. But measures do NOT have row context by default. If you need row-by-row calculation in a measure, use SUMX(Sales, Sales[Amount] * 0.20).

SECTION 8 | Iterator Functions (X-Functions)

Iterator functions are special DAX functions that end with the letter "X" (like SUMX, AVERAGEX, COUNTX). What makes them special is that they work row by row. Regular aggregation functions like SUM() look at an entire column at once. But SUMX() goes through each row one by one, calculates something for that row, and then adds up all the results.

How Iterators Work

SUMX(Table, Expression)

-- Step 1: Go to Row 1, evaluate Expression for Row 1

-- Step 2: Go to Row 2, evaluate Expression for Row 2

-- Step 3: ... continue for all rows ...

-- Step 4: Add up all the results

All Iterator Functions

Function	What It Does	Example
SUMX()	Adds up a row-by-row calculation	SUMX(Sales, Sales[Price] * Sales[Qty])
AVERAGEX()	Average of a row-by-row calculation	AVERAGEX(Sales, Sales[Revenue] - Sales[Cost])
MINX()	Minimum of a row-by-row calculation	MINX(Sales, Sales[Amount] * 0.9)
MAXX()	Maximum of a row-by-row calculation	MAXX(Sales, Sales[Price] * Sales[Discount])
COUNTX()	Counts results of a row-by-row calculation	COUNTX(Sales, Sales[Product])
CONCATENATEX()	Joins text from rows	CONCATENATEX(VALUES(Product[Name]), [Name], ", ")
RANKX()	Ranks items based on an expression	RANKX(ALL(Customer), [Total Sales], [Total Sales], DESC)

Real Example: Profit Calculation

-- WRONG (SUM cannot multiply columns row by row):

Total Profit Wrong = SUM(Sales[Price]) - SUM(Sales[Cost])

-- CORRECT (SUMX multiplies row by row first):

Total Profit Correct = SUMX(Sales, Sales[Price] - Sales[Cost])

When to Use X-Functions

Use SUM, AVERAGE, MIN, MAX when you only need one column. Use SUMX, AVERAGEX when your calculation involves multiple columns or conditional logic per row.

SECTION 9 | Data Modeling (Star Schema)

Data modeling is the process of organizing your tables and relationships so that Power BI works efficiently. A good data model makes your reports fast, accurate, and easy to build. A bad data model leads to slow reports, wrong numbers, and confusing visuals. The Star Schema is the best practice for Power BI data modeling.

Fact Table vs Dimension Table

Aspect	Fact Table	Dimension Table
Contains	Numbers, measurements, transactions	Descriptions, categories, attributes
Rows	Many rows (thousands or millions)	Fewer rows (hundreds or thousands)
Example	Sales: OrderID, Date, ProductID, Amount, Qty, Cost	Product: ProductID, Name, Category, Color, Brand
Relationship	Many-to-one with Dimensions	One-to-many with Fact

Relationship Rules

- Use Single direction (one-to-many) relationships. Filter goes from Dimension (one side) to Fact (many side).
- The Dimension table is on the "one" side (has the unique ID). The Fact table is on the "many" side (has the foreign key).
- Avoid bi-directional relationships unless you really need them. They can cause confusion and slow performance.

Interview Tip

"Why is Star Schema the best practice?" Answer: "Star Schema is best because it gives clear separation between facts and dimensions, makes DAX calculations simpler, improves report performance, and is easy for other developers to understand and maintain."

SECTION 10 | Date Table

A Date Table is a special table that contains one row for every date in your data range. It has columns for Year, Month, Quarter, Week, and Day Name. Every professional Power BI report needs a Date Table because many DAX functions (especially Time Intelligence functions) require it.

How to Create a Date Table

```
DimDate =  
ADDCOLUMNS(  
    CALENDAR(DATE(2020,1,1), DATE(2025,12,31)),  
    "Year", YEAR([Date]),  
    "Month", MONTH([Date]),  
    "MonthName", FORMAT([Date], "MMMM"),  
    "Quarter", "Q" & CEILING(MONTH([Date])/3, 1),  
    "YearQuarter", "Q" & CEILING(MONTH([Date])/3, 1) & " " & YEAR([Date]),  
    "WeekNum", WEEKNUM([Date]),  
    "DayName", FORMAT([Date], "dddd"),  
    "IsWeekend", IF(WEEKDAY([Date],2) >= 6, "Yes", "No"),  
    "YearMonth", FORMAT([Date], "YYYY-MM")  
)
```

How to Mark as Date Table

- Step 1: Click on your Date Table
- Step 2: On the top ribbon, click "Mark as Date Table"
- Step 3: Select the Date column as the date identifier

Important

Your Date Table must have continuous dates (no gaps). Every single date from start to end must exist. Also, the date column must be a Date/Time data type.

SECTION 11 | 15 Interview Q&A

These are the most frequently asked Power BI interview questions. Study them carefully.

Q1: What is Power BI?

Power BI is a business intelligence tool by Microsoft. It connects to data sources, transforms data with Power Query, builds interactive reports and dashboards, and shares them online. It has three parts: Power BI Desktop (free), Power BI Service (cloud), and Power BI Mobile (app).

Q2: What is the difference between Power BI Desktop and Power BI Service?

Power BI Desktop is the free application where you build reports on your computer. Power BI Service is the cloud platform where you publish, share, and manage reports. You build in Desktop, share in Service.

Q3: What is DAX?

DAX stands for Data Analysis Expressions. It is the formula language used in Power BI to create custom calculations. You can build measures and calculated columns with DAX. It is more powerful than Excel formulas because it works with entire tables and has concepts like filter context.

Q4: What is the difference between a Measure and a Calculated Column?

A Calculated Column adds a new column to a table and calculates a value for each row. It uses more memory. A Measure is a dynamic calculation that changes based on filters and user selections. It uses less memory because it calculates on demand. Always prefer Measures.

Q5: What is CALCULATE and why is it important?

CALCULATE is the most important DAX function. It is the only function that can modify the filter context. It takes an expression and one or more filter conditions, then calculates the expression in the modified context.

Q6: What is Filter Context?

Filter Context is the set of filters that determine which rows are included in a calculation. Filters come from slicers, visual filters, page filters, and CALCULATE. It answers: "Which data should I look at?"

Q7: What is Row Context?

Row Context exists when DAX looks at one row at a time. This happens in Calculated Columns and in iterator functions like SUMX and FILTER. It answers: "For this specific row, what is the value?"

Q8: What is a Star Schema?

Star Schema is a data model design where one central Fact Table (with numbers) is connected to several Dimension Tables (with descriptions). It looks like a star. It is the best practice because it makes DAX simpler and reports faster.

Q9: What is a Date Table and why do you need it?

A Date Table is a table with one row per date and columns for Year, Month, Quarter, etc. You need it because Time Intelligence DAX functions (like TOTALYTD, SAMEPERIODLASTYEAR) require a proper Date Table.

Q10: What is the difference between SUM and SUMX?

SUM adds all values in a single column. SUMX goes row by row through a table, evaluates an expression for each row, and then adds up all results. Use SUM for simple column totals. Use SUMX for calculations involving multiple columns.

Q11: What is Power Query?

Power Query is the data transformation tool inside Power BI. It uses M language behind the scenes. You use it to clean data: remove columns, filter rows, split text, merge tables. Power Query runs BEFORE the data loads into the model.

Q12: What is the difference between DirectQuery and Import mode?

In Import mode, Power BI copies all data into its own memory. Reports are fast but data needs refresh. In DirectQuery, Power BI queries the source database directly. Data is always live but may be slower.

Q13: What is a relationship cardinality?

Cardinality defines how tables relate: Many-to-One (one product has many sales), One-to-One (one employee has one badge), or Many-to-Many (special cases). Most relationships are Many-to-One.

Q14: What are incremental refresh and its benefits?

Incremental refresh means Power BI only refreshes new or changed data instead of the entire dataset. This makes refresh faster, uses less memory, and allows working with larger datasets.

Q15: How do you optimize a slow Power BI report?

I would: (1) Remove unnecessary columns and tables, (2) Use Star Schema, (3) Use Measures instead of Calculated Columns, (4) Avoid bi-directional relationships, (5) Use Aggregations for large tables, (6) Reduce visual count per page, (7) Use Performance Analyzer.

SECTION 12 | Quick Tips & Shortcuts

Keyboard Shortcuts

Shortcut	What It Does
Ctrl + S	Save your Power BI file (.pbix)
Ctrl + Z	Undo last action
Ctrl + Y	Redo (undo the undo)
Ctrl + D	Duplicate a visual on the report page
Alt + Click	Select multiple visuals at once
Ctrl + Click	Select multiple visuals (add to selection)
Esc	Exit editing mode / deselect all

Performance Tips

- Hide unused columns: Right-click and "Hide in report view." This reduces file size.
- Use Measures not Columns: Measures calculate on demand. Columns store data.
- Avoid bi-directional filters: Use single direction (Dimension to Fact).
- Use Performance Analyzer: Go to View > Performance Analyzer.

DAX Tips

- Use variables (VAR): Makes DAX readable and sometimes faster:

```
Total Profit =  
VAR Revenue = SUMX(Sales, Sales[Qty] * Sales[Price])  
VAR Cost = SUMX(Sales, Sales[Qty] * Sales[UnitCost])  
RETURN Revenue - Cost
```

- Always use DIVIDE instead of / : DIVIDE handles division by zero errors:

```
Ratio = DIVIDE([Numerator], [Denominator], 0)
```

SECTION 13 | DAX Patterns

DAX Patterns are reusable formulas that solve common business problems. Learn these and you can build powerful reports quickly.

Pattern 1: Year-to-Date

```
Total YTD = TOTALYTD(SUM(Sales[Amount]), Date[Date])
```

Pattern 2: Same Period Last Year

```
Sales SPLY = CALCULATE(  
SUM(Sales[Amount]),  
SAMEPERIODLASTYEAR(Date[Date])  
)
```

Pattern 3: YoY Growth

```
YoY Growth = DIVIDE(  
[Total YTD] - [Sales SPLY],  
[Sales SPLY],  
0  
)
```

Pattern 4: Moving Average

```
7-Day Avg = AVERAGEX(  
DATESINPERIOD(Date[Date], LASTDATE(Date[Date]), -7, DAY),  
[Daily Sales]  
)
```

Pattern 5: Rank Products

```
Product Rank = RANKX(  
ALL(Product[Name]),  
[Total Sales],  
[Total Sales],  
DESC  
)
```

SECTION 14 | Quick Reference Card

DAX Essentials

Concept	Key Fact
DAX stands for	Data Analysis Expressions
Two calculation types	Measures (dynamic) and Calculated Columns (row-by-row)
Most important function	CALCULATE (only function that modifies filter context)
Two contexts	Filter Context (which rows) and Row Context (one row at a time)
X-Functions	SUMX, AVERAGEX, etc. Loop row by row, then aggregate
Best practice	Always prefer Measures over Calculated Columns
Division	Always use DIVIDE() instead of / to handle zero division
Variables	Use VAR and RETURN to make DAX readable and fast
Date Table	Required for Time Intelligence. Must have continuous dates.
Star Schema	Fact Table (numbers) in center, Dimension Tables around it

Top 10 DAX Functions

Function	Category	Purpose
CALCULATE	Filter	Modify filter context for a measure
SUMX	Iterator	Row-by-row sum with expression
FILTER	Filter	Return a table with filtered rows
ALL	Filter	Remove all filters from a column or table
RELATED	Relationship	Get value from a related table
TOTALYTD	Time Intel	Year-to-date total
SAMEPERIODLASTYEAR	Time Intel	Same dates in previous year
DIVIDE	Math	Safe division (handles zero)
DISTINCTCOUNT	Aggregation	Count unique values
IF / SWITCH	Logical	Conditional logic

SECTION 15 | Professional Workflow Sequence

This section shows you the exact step-by-step process that professional Power BI developers follow on real projects. If you can explain this workflow, you will stand out in interviews.

STEP 1: Understand the Business Requirements

Before opening Power BI, ask questions: What business questions need answering? Who will use this report? What data sources are available? What is the deadline? Deliverable: A requirements document.

STEP 2: Explore and Profile the Data

Look at the actual data. How many tables? How many rows? Are there missing values? Are dates in the correct format? Are there data quality issues?

STEP 3: Data Transformation with Power Query

Clean the data: Remove unnecessary columns, filter rows, fix data types, handle missing values, merge tables, rename columns, remove duplicates.

STEP 4: Build the Data Model (Star Schema)

Create the Data Table, connect all tables, set correct cardinality (many-to-one), filter direction (single), hide unnecessary columns.

STEP 5: Write DAX Measures

Start with basic measures (Total Sales, Profit, Margin), then add Time Intelligence (YTD, SPLY, YoY Growth), then rankings. Test each measure in a table visual.

STEP 6: Design the Report Pages

Page 1 = Dashboard (KPIs, key charts). Page 2 = Details. Page 3 = Analysis. Use consistent formatting, add slicers, use bookmarks for navigation.

STEP 7: Testing and Quality Check

Do total numbers match the source? Do all filters work? Any DAX errors? Is the report fast? Use Performance Analyzer.

STEP 8: Publish and Share

Publish to Power BI Service, set up data refresh, share with stakeholders, set Row-Level Security if needed, write documentation.

Summary

The workflow: Requirements > Data Exploration > Power Query > Data Model > DAX Measures > Report Design > Testing > Publish. Never skip the first 3 steps!

SECTION 16 | Free Power BI Desktop Installation Tutorial

Power BI Desktop is 100% free. Follow these steps to install and start learning.

System Requirements

Requirement	Minimum	Recommended
Operating System	Windows 10 (64-bit)	Windows 11 (64-bit)
Memory (RAM)	4 GB	8 GB or more
Disk Space	2 GB	10 GB free space
Screen	1280 x 720	1920 x 1080

Download and Install

STEP 1: Download

Go to: www.microsoft.com/en-us/download/details.aspx?id=58494 or search "Download Power BI Desktop Free". Click the blue Download button. Choose "Power BI Desktop (x64)."

STEP 2: Install

Open the downloaded file (PBIDesktopSetup.exe). Click Next > Accept > Next > Install > Finish. Power BI Desktop will open automatically.

STEP 3: Your First Chart

Go to Home > Get Data > Excel > select your file > Load. Click Report view. From Visualizations, click Clustered Bar Chart. Drag a text field to Axis and a number field to Values. Your first chart is ready!

Learning Path

Priority	What to Learn	How Long
1	Power Query basics: connect, clean, transform	1-2 weeks
2	Build simple reports with basic visuals	1 week
3	Data modeling: relationships, star schema	1-2 weeks
4	DAX basics: measures, simple calculations	2-3 weeks
5	DAX advanced: CALCULATE, Time Intelligence	2-3 weeks
6	Publishing and sharing	1 week
7	Advanced: M language, performance tuning, RLS	Ongoing

SECTION 17 | Realistic Data Sources for Practice

To become good at Power BI, you need to practice with real data. Here are the best free sources.

Beginner (Easy, Small Datasets)

Source	What You Get	How to Access
Microsoft Sample Datasets	Excel files for Power BI learning	Get Data > Sample Reports in Power BI Desktop
Adventure Works	Sample database with Sales, Products, Customers	Search "Adventure Works sample database" on Microsoft docs
Kaggle Datasets	Thousands of free datasets from real companies	www.kaggle.com/datasets

Intermediate (Medium Datasets)

Source	What You Get	How to Access
Data.gov	US government open data: population, economy, health	www.data.gov
World Bank Open Data	GDP, population, education for every country	data.worldbank.org
WHO Data	Health statistics globally	www.who.int/data

Advanced (Large, Real-World)

Source	What You Get	How to Access
NYC Open Data	1 billion+ taxi rides, 311 complaints, crime data	data.cityofnewyork.us
GitHub Public Data	Millions of code repositories	github.com
COVID-19 Data	Cases, deaths, vaccinations by country	github.com/CSSEGISandData/COVID-19

SECTION 18 | Best Practices & Advanced Approach

These are the rules, standards, and methods that professional Power BI developers follow. Following these rules makes your reports faster, cleaner, and more reliable.

18.1 Data Modeling Best Practices

- Always use Star Schema: One Fact Table connected to Dimension Tables. Avoid snowflake schemas and flat single-table models.
- Fact Table = Numbers Only: The Fact Table should contain only keys (IDs) and numeric measures.
- Dimension Table = One Row Per Entity: Each Dimension has exactly one row per unique entity.
- Single Direction Relationships: Filter direction from Dimension (one) to Fact (many).
- Hide Foreign Key Columns: Hide ID columns in the Fact Table from report view.

Anti-Pattern: Single Flat Table

Do NOT put all data into one big flat table. It makes DAX complicated, causes filter problems, and produces slow reports. Always split into Facts and Dimensions.

18.2 DAX Best Practices

- Use Measures, Not Calculated Columns: Measures use less memory and give more flexible results.
- Always Use DIVIDE(): `DIVIDE(Measure1, Measure2, 0)` returns 0 instead of an error when denominator is zero.
- Use Variables (VAR): Break complex DAX into steps. Makes code readable and debuggable.
- Name Measures Clearly: Use descriptive names: "Total Sales", "Avg Order Value", "YTD Revenue."
- Test in Simple Table First: Before putting a measure in a complex chart, test it in a table visual.

-- GOOD DAX with variables:

Profit Margin % =

```
VAR TotalRevenue = SUMX(Sales, Sales[Qty] * Sales[Price])
```

```
VAR TotalCost = SUMX(Sales, Sales[Qty] * Sales[UnitCost])
```

```
VAR Profit = TotalRevenue - TotalCost
```

```
RETURN DIVIDE(Profit, TotalRevenue, 0)
```

18.3 Power Query Best Practices

- Clean in Power Query, not DAX: Power Query runs once. DAX runs every time a user interacts.
- Remove unused columns early: Every column takes memory.
- Replace blanks with values: Null values cause DAX errors. Replace with 0 for numbers.

18.4 Report Design Best Practices

- KPIs at the top: Put key numbers in large card visuals at the top of the dashboard.
- Limit 5-8 visuals per page: Too many visuals confuse users and slow the report.
- Consistent colors: Choose 3-5 colors and use them consistently.

- Add slicers: Place them at the top or left for easy filtering.
- Logical page flow: Summary > Details > Analysis.

18.5 Performance Optimization

Rule	Why	How
Remove unused columns	Saves memory	Right-click > Hide in report view
Remove unused tables	Saves memory	Delete unconnected tables
Use Import mode for small data	Faster visuals	Choose Import for data under 1 GB
Use DirectQuery for large data	Handles big datasets	Choose DirectQuery for millions of rows
Limit visual complexity	Each visual sends a query	5-8 visuals per page max

18.6 Naming Conventions

Object	Convention	Example
Tables (Fact)	Prefix "Fact"	FactSales, FactOrders
Tables (Dimension)	Prefix "Dim"	DimProduct, DimCustomer, DimDate
Measures	Descriptive name	Total Revenue, Avg Order Value
Columns	PascalCase	SalesAmount, OrderDate
Pages	Numbered + descriptive	01_Dashboard, 02_Details

18.7 Error Handling

- Use DIVIDE with default: DIVIDE([A], [B], 0) returns 0 when B is zero.
- Use ISBLANK: IF(ISBLANK([Measure]), 0, [Measure]) ensures no blank values.
- Use COALESCE: COALESCE([M1], [M2], 0) tries first, then second, then returns 0.

18.8 Development Path

Level	Timeline	What to Learn
Beginner	1-2 months	Basics, connect to Excel, basic charts, simple DAX
Intermediate	3-4 months	Star Schema, Power Query, CALCULATE, Time Intel
Advanced	5-8 months	Complex DAX, performance tuning, RLS, M language
Professional	8-12 months	Enterprise features, deployment, governance
Expert	1+ year	Custom visuals, R/Python, Azure, mentoring, PL-300 cert

Interview Tip

If someone asks about best practices, mention: Star Schema, Measures over Columns, DIVIDE instead of /, VAR for readability, Power Query for cleaning, consistent naming, Performance Analyzer, and Row-Level Security.